

Black–Scholes PDE Solver

Mathematical Documentation

A complete exposition of all numerical methods, discretisations, stability analyses, and convergence results implemented in the codebase

RobinBasso

October 2, 2026

Abstract

This document provides a rigorous mathematical account of every computation performed in the `black-scholes-pde` project. The codebase implements three finite difference schemes for the Black–Scholes partial differential equation (PDE): an explicit (forward Euler in time) scheme, a Crank–Nicolson scheme, and Crank–Nicolson with Rannacher smoothing. All methods are validated against the closed-form Black–Scholes formula and subjected to systematic spatial and temporal convergence studies. Every formula, coefficient, matrix, stability condition, and convergence result stated in this document corresponds directly to the source code in `src/solver.py`, `src/utils.py`, and `tests/convergence.py`.

Contents

1	Notation and Conventions	2
2	The Black–Scholes PDE	2
2.1	Model and Equation	2
2.2	Terminal Condition	2
2.3	Boundary Conditions	3
3	Analytical Solution (<code>utils.py</code>)	3
3.1	Black–Scholes Closed-Form Formula	3
4	Spatial Discretisation	4
4.1	Grid Construction	4
4.2	Finite Difference Operators	4
4.3	Substitution into the PDE	4
5	Explicit Finite Difference Scheme (<code>solve_explicit</code>)	4
5.1	Time Discretisation	4
5.2	Coefficients	5
5.3	Stability Analysis	5
5.4	Truncation Error	5

6	Crank–Nicolson Scheme (<code>solve_crank_nicolson</code>)	6
6.1	Time Discretisation	6
6.2	Coefficient Derivation	6
6.3	Linear System	6
6.4	Boundary Corrections	7
6.5	Linear Solve	7
6.6	Truncation Error and Stability	7
7	Rannacher Smoothing (<code>solve_rannacher</code>)	8
7.1	Motivation: Payoff Regularity	8
7.2	The Rannacher Procedure	8
7.3	Implicit Euler Coefficients	8
7.4	Right-Hand Side During Implicit Euler Phase	9
7.5	Truncation Error	9
8	Convergence Analysis (<code>tests/convergence.py</code>)	9
8.1	Error Metric	9
8.2	Spatial Convergence Study	10
8.3	Time Convergence Study	10
8.4	Time Convergence with Rannacher Smoothing	10
9	End-to-End Computational Summary	11
10	Scheme Comparison	11
11	Module Architecture Summary	12
12	Limitations and Potential Extensions	12

1 Notation and Conventions

Symbol	Meaning
S	Underlying asset price
t	Time (calendar time, $t \in [0, T]$)
$\tau = T - t$	Time to maturity (backward time)
$V(S, t)$	Option price as a function of S and t
K	Strike price (default: 100)
T	Maturity (default: 1.0 year)
r	Risk-free interest rate (default: 0.05)
σ	Volatility (default: 0.20)
$S_{\max}$	Truncation of the spatial domain (default: 200)
M	Number of spatial intervals ($M + 1$ grid points)
N	Number of time steps
$\Delta S = S_{\max}/M$	Spatial step size
$\Delta t = T/N$	Time step size
$S_i = i \Delta S$	i -th spatial grid point, $i = 0, 1, \dots, M$
$t^n = n \Delta t$	n -th time level (forward), $n = 0, 1, \dots, N$
V_i^n	Numerical approximation to $V(S_i, t^n)$

The code solves the PDE **backwards in time** from $t = T$ (terminal condition) to $t = 0$ (option price today), storing the solution on a grid `grid[i, j]` where the column index j runs from 0 (today) to N (maturity). The analytical solution is stored at column $j = 0$ after backward stepping.

2 The Black–Scholes PDE

2.1 Model and Equation

Under the Black–Scholes model, the no-arbitrage price $V(S, t)$ of a European option satisfies the PDE:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0, \quad (S, t) \in (0, S_{\max}) \times [0, T].} \quad (1)$$

The three terms beyond $\partial V/\partial t$ have clear financial interpretations:

- $\frac{1}{2}\sigma^2 S^2 \partial^2 V/\partial S^2$: convexity / gamma term (diffusion).
- $rS \partial V/\partial S$: delta term (advection).
- $-rV$: discounting (reaction / zeroth-order term).

2.2 Terminal Condition

For a European call option with strike K :

$$V(S, T) = \max(S - K, 0). \quad (2)$$

This is the **payoff function**. In the code it is initialised as:

```
1 grid[:, -1] = np.maximum(S - K, 0)
```

where the last column ($j = N$) corresponds to $t = T$.

2.3 Boundary Conditions

Lower boundary ($S = 0$). When the asset price is zero, it stays zero under the geometric Brownian motion model, so the call option is worthless:

$$V(0, t) = 0 \quad \forall t \in [0, T]. \quad (3)$$

Upper boundary ($S = S_{\max}$). For large S , the probability of the option expiring out-of-the-money approaches zero and the option behaves like a forward contract. The standard approximation is:

$$V(S_{\max}, t) \approx S_{\max} - Ke^{-r(T-t)}. \quad (4)$$

In the code, this is precomputed over all time levels:

```
1 time = np.linspace(0, T, N + 1)
2 grid[-1, :] = S_max - K * np.exp(-r * (T - time))
```

3 Analytical Solution (utils.py)

3.1 Black–Scholes Closed-Form Formula

The exact price of a European call under the Black–Scholes model is:

$$V(S, t) = S \Phi(d_1) - Ke^{-r(T-t)} \Phi(d_2) \quad (5)$$

where Φ is the standard normal CDF and:

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)(T - t)}{\sigma\sqrt{T - t}}, \quad (6)$$

$$d_2 = d_1 - \sigma\sqrt{T - t}. \quad (7)$$

The implementation in `black_scholes_call` evaluates this at $t = 0$ (i.e., $T - t = T$), masking out $S = 0$ to avoid $\ln(0)$:

```
1 mask = S > 0
2 d1 = (np.log(S_pos / K) + (r + 0.5*sigma**2)*T) / (sigma*np.
    sqrt(T))
3 d2 = d1 - sigma * np.sqrt(T)
4 price[mask] = S_pos * norm.cdf(d1) - K * np.exp(-r*T) * norm.cdf
    (d2)
```

This serves as the **ground truth** for all error computations in the project.

4 Spatial Discretisation

4.1 Grid Construction

The spatial domain $[0, S_{\max}]$ is divided into M equal intervals:

$$S_i = i \Delta S, \quad \Delta S = \frac{S_{\max}}{M}, \quad i = 0, 1, \dots, M.$$

The grid has $M + 1$ points total. Interior points are $i = 1, \dots, M - 1$; boundary points are $i = 0$ and $i = M$.

4.2 Finite Difference Operators

The spatial derivatives in equation (1) are approximated by **second-order central differences**:

$$\left. \frac{\partial V}{\partial S} \right|_{S=S_i} \approx \frac{V_{i+1} - V_{i-1}}{2 \Delta S}, \quad (8)$$

$$\left. \frac{\partial^2 V}{\partial S^2} \right|_{S=S_i} \approx \frac{V_{i+1} - 2V_i + V_{i-1}}{\Delta S^2}. \quad (9)$$

Both are $O(\Delta S^2)$ accurate, which underpins the second-order spatial convergence observed in all three schemes.

4.3 Substitution into the PDE

Substituting (8)–(9) into (1), noting that $S_i = i \Delta S$ so $S_i / \Delta S = i$, and using $S_i^2 / \Delta S^2 = i^2$, yields:

$$\frac{\partial V_i}{\partial t} + \frac{\sigma^2 i^2}{2} (V_{i+1} - 2V_i + V_{i-1}) + \frac{ri}{2} (V_{i+1} - V_{i-1}) - rV_i = 0. \quad (10)$$

This is a *semi-discrete* system (continuous in time, discrete in space), from which the fully discrete schemes are derived by approximating $\partial V_i / \partial t$.

5 Explicit Finite Difference Scheme (`solve_explicit`)

5.1 Time Discretisation

The time derivative is approximated by a **forward (explicit) Euler** step but since we march *backwards* from $t = T$ to $t = 0$, the update reads:

$$\frac{\partial V_i}{\partial t} \approx \frac{V_i^{n+1} - V_i^n}{\Delta t} \implies V_i^n = V_i^{n+1} + \Delta t [\text{spatial terms at } n + 1].$$

Substituting into (10) and rearranging:

$$\boxed{V_i^n = a_i V_{i-1}^{n+1} + b_i V_i^{n+1} + c_i V_{i+1}^{n+1}} \quad (11)$$

5.2 Coefficients

$$a_i = \frac{1}{2} \Delta t (\sigma^2 i^2 - r i), \quad (12)$$

$$b_i = 1 - \Delta t (\sigma^2 i^2 + r), \quad (13)$$

$$c_i = \frac{1}{2} \Delta t (\sigma^2 i^2 + r i). \quad (14)$$

These match exactly the code:

```
1 a = 0.5 * dt * (sigma**2 * i**2 - r * i)
2 b = 1 - dt * (sigma**2 * i**2 + r)
3 c = 0.5 * dt * (sigma**2 * i**2 + r * i)
4 grid[i, j] = a * grid[i-1, j+1] + b * grid[i, j+1] + c * grid[i
  +1, j+1]
```

Note that $a_i + b_i + c_i = 1 - \Delta t \cdot r$, which is less than 1 for $r > 0$, consistent with discounting.

5.3 Stability Analysis

The explicit scheme is **conditionally stable**. The stability condition requires that all coefficients a_i , b_i , c_i be non-negative (a sufficient condition for monotonicity / maximum principle):

$$b_i \geq 0 \iff \Delta t \leq \frac{1}{\sigma^2 i^2 + r}.$$

The most restrictive constraint is at $i = M$ (largest S):

$$\Delta t \lesssim \frac{1}{\sigma^2 M^2} \quad (15)$$

This is a **CFL-type condition**. In the convergence study, the explicit scheme is given $N = 2M^2$ time steps to ensure stability:

```
1 N_explicit = M * M * 2 # ensures dt << 1 / (sigma^2 * M^2)
```

Violation of (15) causes the scheme to diverge, as demonstrated in the time convergence tests.

5.4 Truncation Error

The local truncation error (LTE) of the explicit scheme is:

$$\tau = O(\Delta t) + O(\Delta S^2).$$

Hence the scheme is **first-order in time** and **second-order in space**. When the CFL condition is tight ($\Delta t \sim 1/(\sigma^2 M^2) \sim \Delta S^2$), both errors are of comparable magnitude.

6 Crank–Nicolson Scheme (solve_crank_nicolson)

6.1 Time Discretisation

The Crank–Nicolson method averages the spatial operator at time levels n and $n + 1$:

$$\frac{V_i^n - V_i^{n+1}}{\Delta t} = \frac{1}{2} \mathcal{L}_i[V^n] + \frac{1}{2} \mathcal{L}_i[V^{n+1}], \quad (16)$$

where the spatial operator (from equation (10)) is:

$$\mathcal{L}_i[V] = \frac{\sigma^2 i^2}{2} (V_{i+1} - 2V_i + V_{i-1}) + \frac{ri}{2} (V_{i+1} - V_{i-1}) - rV_i.$$

6.2 Coefficient Derivation

Define half-step coefficients:

$$\alpha_i = \frac{1}{4} \Delta t (\sigma^2 i^2 - ri), \quad (17)$$

$$\beta_i = -\frac{1}{2} \Delta t (\sigma^2 i^2 + r), \quad (18)$$

$$\gamma_i = \frac{1}{4} \Delta t (\sigma^2 i^2 + ri). \quad (19)$$

Note that $\alpha_i = a_i/2$ and $\gamma_i = c_i/2$ compared to the explicit coefficients, and $\beta_i = (b_i - 1)/2$. These match the code:

```
1 alpha = 0.25 * dt * (sigma**2 * i**2 - r * i)
2 beta  = -0.5 * dt * (sigma**2 * i**2 + r)
3 gamma = 0.25 * dt * (sigma**2 * i**2 + r * i)
```

6.3 Linear System

Rearranging (16) moves the unknowns (V^n) to the left and the knowns (V^{n+1}) to the right, giving a tridiagonal linear system for the interior points $i = 1, \dots, M - 1$:

$$\mathbf{A} \mathbf{V}^n = \mathbf{B} \mathbf{V}^{n+1} + \mathbf{b}_{bc}, \quad (20)$$

where $\mathbf{V}^n = (V_1^n, V_2^n, \dots, V_{M-1}^n)^\top$ and the $(M - 1) \times (M - 1)$ tridiagonal matrices are:

Matrix A (implicit side):

$$\mathbf{A} = \begin{pmatrix} 1 - \beta_1 & -\gamma_1 & 0 & \cdots & 0 \\ -\alpha_2 & 1 - \beta_2 & -\gamma_2 & \cdots & 0 \\ 0 & -\alpha_3 & 1 - \beta_3 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & -\alpha_{M-1} & 1 - \beta_{M-1} \end{pmatrix} \quad (21)$$

Matrix B (explicit side):

$$\mathbf{B} = \begin{pmatrix} 1 + \beta_1 & \gamma_1 & 0 & \cdots & 0 \\ \alpha_2 & 1 + \beta_2 & \gamma_2 & \cdots & 0 \\ 0 & \alpha_3 & 1 + \beta_3 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & \alpha_{M-1} & 1 + \beta_{M-1} \end{pmatrix} \quad (22)$$

Note that $\mathbf{A}$ and $\mathbf{B}$ satisfy $\mathbf{A} + \mathbf{B} = 2\mathbf{I}$, up to boundary corrections. In the code:

```
1 A[row, row] = 1 - beta; B[row, row] = 1 + beta
2 A[row, row-1] = -alpha; B[row, row-1] = alpha
3 A[row, row+1] = -gamma; B[row, row+1] = gamma
```

6.4 Boundary Corrections

The right-hand side vector $\mathbf{b}_{bc}$ accounts for the boundary values at $i = 0$ and $i = M$ which appear in the stencil for the first and last interior rows. For row $i = 1$ (lower boundary):

$$[\mathbf{b}_{bc}]_1 = \alpha_1(V_0^n + V_0^{n+1}) = \alpha_1 \cdot 0 = 0,$$

but since $V_0^n = 0$ always, the correction is zero for the lower boundary. For row $i = M - 1$ (upper boundary):

$$[\mathbf{b}_{bc}]_{M-1} = \gamma_{M-1}(V_M^n + V_M^{n+1}).$$

This is added in the time loop:

```
1 # Lower boundary (i=1): V_0 = 0, so correction is zero
2 alpha_1 = 0.25 * dt * (sigma**2 * 1**2 - r * 1)
3 rhs[0] += alpha_1 * (grid[0, j] + grid[0, j+1])
4
5 # Upper boundary (i=M-1)
6 gamma_Mm1 = 0.25 * dt * (sigma**2 * (M-1)**2 + r * (M-1))
7 rhs[-1] += gamma_Mm1 * (grid[M, j] + grid[M, j+1])
```

6.5 Linear Solve

At each time step, the system $\mathbf{A}\mathbf{x} = \mathbf{rhs}$ is solved using `numpy.linalg.solve`, which applies LU decomposition. Since $\mathbf{A}$ is a fixed tridiagonal matrix (time-independent coefficients), it could be pre-factorised for efficiency; the current code recomputes the solve at each step.

6.6 Truncation Error and Stability

The Crank–Nicolson scheme has:

- **Truncation error:** $O(\Delta t^2) + O(\Delta S^2)$ second-order in both.

- **Stability: unconditionally stable** (for the diffusion-dominated case), because $\mathbf{A}$ is strictly diagonally dominant for $\beta_i < 0$, which holds whenever $r > 0$.

The unconditional stability means N can be set equal to M (rather than M^2) in the convergence study:

```
1 N_cn = M      # No CFL constraint
```

7 Rannacher Smoothing (solve_rannacher)

7.1 Motivation: Payoff Regularity

The call payoff $\max(S - K, 0)$ is **continuous but not differentiable** at $S = K$:

$$\left. \frac{\partial V}{\partial S} \right|_{S=K^+} = 1, \quad \left. \frac{\partial V}{\partial S} \right|_{S=K^-} = 0.$$

This kink in the initial condition degrades the temporal convergence of Crank–Nicolson from theoretical $O(\Delta t^2)$ to approximately $O(\Delta t)$ in practice, because the averaging of the operator at two time levels amplifies the discontinuity in the derivative.

7.2 The Rannacher Procedure

The Rannacher method (1984) applies **fully implicit (backward) Euler** for the first n_{smooth} time steps (default: 4) immediately after the terminal condition, before switching to Crank–Nicolson:

$$\text{Method at step } j = \begin{cases} \text{Implicit Euler} & \text{if } j \geq N - n_{\text{smooth}} \\ \text{Crank–Nicolson} & \text{otherwise} \end{cases} \quad (23)$$

The implicit Euler steps damp the oscillations caused by the non-smooth payoff; subsequent Crank–Nicolson steps then inherit a smooth solution.

7.3 Implicit Euler Coefficients

The implicit Euler discretisation is:

$$\frac{V_i^n - V_i^{n+1}}{\Delta t} = \mathcal{L}_i[V^n],$$

which gives the linear system $\mathbf{A}_{\text{imp}} \mathbf{V}^n = \mathbf{V}^{n+1}$ with tridiagonal matrix coefficients:

$$\alpha_i^{(\text{IE})} = \frac{1}{2} \Delta t (\sigma^2 i^2 - ri), \quad (24)$$

$$\beta_i^{(\text{IE})} = -\Delta t (\sigma^2 i^2 + r), \quad (25)$$

$$\gamma_i^{(\text{IE})} = \frac{1}{2} \Delta t (\sigma^2 i^2 + ri). \quad (26)$$

Note that $\alpha_i^{(\text{IE})} = 2\alpha_i$ and $\gamma_i^{(\text{IE})} = 2\gamma_i$ exactly twice the Crank–Nicolson half-coefficients. This matches the code:

```

1 alpha_e = 0.5 * dt * (sigma**2 * i**2 - r * i)      # = 2 *
   alpha_CN
2 beta_e   = -dt * (sigma**2 * i**2 + r)              # = 2 *
   beta_CN
3 gamma_e  = 0.5 * dt * (sigma**2 * i**2 + r * i)     # = 2 *
   gamma_CN
4
5 A_imp[row, row-1] = -alpha_e
6 A_imp[row, row]   = 1 - beta_e
7 A_imp[row, row+1] = -gamma_e

```

7.4 Right-Hand Side During Implicit Euler Phase

During the Rannacher smoothing phase ($j \geq N - n_{\text{smooth}}$), the RHS for the implicit Euler step is simply V^{n+1} with boundary corrections from both boundaries:

```

1 rhs = grid[1:M, j+1].copy()
2 rhs[0] += 0.5*dt*(sigma**2*1**2 - r*1) * (grid[0, j] + grid
   [0, j+1])
3 rhs[-1] += 0.5*dt*(sigma**2*(M-1)**2 + r*(M-1)) * (grid[M, j] +
   grid[M, j+1])

```

7.5 Truncation Error

- Implicit Euler: $O(\Delta t) + O(\Delta S^2)$ first-order in time.
- Crank–Nicolson: $O(\Delta t^2) + O(\Delta S^2)$ second-order in time (theoretical).

With the Rannacher procedure, the observed convergence is **approximately first-order in time** rather than second-order, because the $O(\Delta t)$ error from the initial smoothing steps dominates. However, the smoothing eliminates the spurious oscillations that would otherwise completely destroy the convergence rate of plain Crank–Nicolson on non-smooth payoffs.

8 Convergence Analysis (tests/convergence.py)

8.1 Error Metric

All convergence studies use the **maximum absolute error** (sup-norm) over the spatial grid at $t = 0$:

$$\mathcal{E} = \max_{i=0,\dots,M} |V_i^{\text{num}} - V^{\text{exact}}(S_i)|, \quad (27)$$

implemented as:

```
1 error = np.max(np.abs(price_numerical - price_exact))
```

8.2 Spatial Convergence Study

Setup. Spatial resolution is varied: $M \in \{25, 50, 100, 200\}$. For each M :

- Crank–Nicolson: $N = M$ (time error negligible, dominated by space).
- Explicit: $N = 2M^2$ (ensures CFL stability without time error dominating).

Expected result. Both schemes use second-order central differences in space, giving:

$$\mathcal{E} = O(\Delta S^2) = O(M^{-2}).$$

Observed result. Both schemes track the $O(M^{-2})$ reference slope on a log-log plot. However, neither exactly achieves the theoretical rate: the kink in the payoff at $S = K$ locally reduces the spatial regularity and slightly degrades the observed order.

8.3 Time Convergence Study

Setup. Fixed $M = 50$ (spatial error fixed). The stability limit is:

$$\Delta t_{\text{stab}} = \frac{1}{\sigma^2 M^2} = \frac{1}{0.04 \times 2500} = \frac{1}{100} = 0.01, \quad N_{\min} = \left\lfloor \frac{T}{\Delta t_{\text{stab}}} \right\rfloor = 100.$$

Both schemes are run at $N \in \{N_{\min}, 2N_{\min}, 4N_{\min}, 8N_{\min}, 16N_{\min}\}$.

Explicit scheme instability. When $N < N_{\min}$, the explicit scheme violates the CFL condition (15) and diverges. The convergence study shows this by using N values just above the stability boundary.

Saturation phenomenon. Once the explicit scheme is stabilised, increasing N barely reduces the error, because the dominant error is now *spatial* (fixed M). The time error $O(\Delta t) = O(1/N)$ quickly becomes negligible compared to $O(\Delta S^2) = O(M^{-2})$.

Crank–Nicolson. Similar saturation occurs: the scheme is already accurate enough in time for reasonable N , and the spatial error floor is reached quickly.

8.4 Time Convergence with Rannacher Smoothing

Setup. Fixed $M = 200$ (fine spatial grid to suppress spatial error). Time steps $N \in \{100, 200, 400, 800, 1600\}$.

Observed convergence. The Rannacher scheme achieves approximately $O(\Delta t) = O(N^{-1})$ in time, matching the $O(dt)$ reference slope on the log-log plot. This is **better** than plain Crank–Nicolson on the same non-smooth payoff (which stagnates or oscillates), but **worse** than theoretical second-order because the implicit Euler smoothing steps introduce $O(\Delta t)$ error.

Remark 8.1. *The $O(\Delta t)$ convergence of Rannacher is not a failure of the method. It is the correct asymptotic rate given the payoff regularity: the gain over plain Crank–Nicolson lies in the constant being much smaller and in the elimination of oscillatory artefacts.*

9 End-to-End Computational Summary

The full computation for a single numerical solution can be written as:

Step 1 Grid initialisation.

$$S_i = i \frac{S_{\max}}{M}, \quad i = 0, \dots, M; \quad t^n = n \frac{T}{N}, \quad n = 0, \dots, N.$$

Step 2 Terminal condition.

$$V_i^N = \max(S_i - K, 0), \quad i = 0, \dots, M.$$

Step 3 Boundary conditions (all n).

$$V_0^n = 0, \quad V_M^n = S_{\max} - K e^{-r(T-t^n)}.$$

Step 4 Backward time stepping (for $j = N - 1, N - 2, \dots, 0$):

- **Explicit:** $V_i^j = a_i V_{i-1}^{j+1} + b_i V_i^{j+1} + c_i V_{i+1}^{j+1}$, $i = 1, \dots, M - 1$.
- **Crank–Nicolson:** solve $\mathbf{A} \mathbf{V}^j = \mathbf{B} \mathbf{V}^{j+1} + \mathbf{b}_{bc}$.
- **Rannacher:** use implicit Euler for $j \geq N - n_{\text{smooth}}$, then Crank–Nicolson.

Step 5 Output.

$$\text{Return } (S_0, S_1, \dots, S_M) \text{ and } (V_0^0, V_1^0, \dots, V_M^0).$$

10 Scheme Comparison

Table 1: Summary comparison of the three implemented schemes

Scheme	Time order	Space order	Stability	System	N needed
Explicit	$O(\Delta t)$	$O(\Delta S^2)$	Conditional (CFL)	Explicit	$O(M^2)$
Crank–Nicolson	$O(\Delta t^2)$	$O(\Delta S^2)$	Unconditional	Tridiagonal	$O(M)$
Rannacher	$O(\Delta t)^*$	$O(\Delta S^2)$	Unconditional	Tridiagonal	$O(M)$

*Practical rate on non-smooth payoff; theoretical CN rate is $O(\Delta t^2)$.

11 Module Architecture Summary

Table 2: Modules and their mathematical roles

File	Function	Mathematical Content
<code>utils.py</code>	<code>black_scholes_call</code>	Closed-form formula (equations 5–7)
<code>solver.py</code>	<code>solve_explicit</code>	Explicit update (equation 11), coefficients (12)–(14)
	<code>solve_crank_nicolson</code>	Tridiagonal system (20), matrices (21)–(22)
	<code>solve_rannacher</code>	Hybrid IE + CN, coefficients (24)–(26)
<code>convergence.py</code>	<code>convergence_study</code>	Spatial $\mathcal{E}$ vs. M (equation 27)
	<code>time_convergence</code>	Temporal $\mathcal{E}$ vs. N , CFL illustration
	<code>time_convergence_rannacher</code>	Rannacher $\mathcal{E}$ vs. N , $O(\Delta t)$ rate
<code>test_solver.py</code>	<code>run_test</code>	Point-wise comparison of all three methods against analytical

12 Limitations and Potential Extensions

The current implementation is a clean, instructive codebase with a few practical limitations worth noting for production use:

1. **European call only.** The terminal condition and upper boundary are hardcoded for a vanilla call. American options require a free-boundary (complementarity) formulation; exotic payoffs require modified terminal conditions.
2. **Non-uniform grids.** The uniform ΔS grid concentrates grid points uniformly across $[0, S_{\max}]$. A sinh-stretching or log-transform grid would place more points near $S = K$, improving accuracy at the kink and potentially restoring second-order spatial convergence in the convergence constant.
3. **Direct linear solver.** `np.linalg.solve` uses full LU decomposition, $O(M^3)$ per time step. For tridiagonal systems, the Thomas algorithm is $O(M)$; using `scipy.linalg.solve_banded` or `scipy.sparse` would give a significant speedup for large M .
4. **Constant $\mathbf{A}$.** The matrix $\mathbf{A}$ is time-independent (coefficients do not depend on t^n). It is rebuilt and refactorised at each time step in the current code; a single pre-factorisation would reduce computational cost.
5. **Single asset, single factor.** The code solves a one-dimensional PDE. Multi-asset options (basket, spread) require higher-dimensional PDE solvers (ADI methods, Monte Carlo).
6. **Fixed parameters.** σ and r are constant. Local volatility models (Dupire) or stochastic volatility models (Heston) introduce $\sigma = \sigma(S, t)$ or additional state variables.

Natural extensions include: Richardson extrapolation for higher-order time accuracy without Rannacher, PSOR (projected SOR) for American options, and a log-space transformation $x = \ln(S/K)$ which converts the PDE to constant-coefficient form and simplifies grid design.

References

- Black, F. & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- Rannacher, R. (1984). Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43(2), 309–327.
- Wilmott, P., Dewynne, J. & Howison, S. (1993). *Option Pricing: Mathematical Models and Computation*. Oxford Financial Press.
- Strikwerda, J. C. (2004). *Finite Difference Schemes and Partial Differential Equations* (2nd ed.). SIAM.
- Seydel, R. U. (2009). *Tools for Computational Finance* (4th ed.). Springer.